

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- ▶ The Encoder-Decoder Model with RNNs
- ▶ Encoder Decoder architecture
- ▶ Training Encoder Decoder
- ▶ Applications of LSTM
- ▶ Attention
- ▶ Summary RNN

The Encoder-Decoder Model with RNNs

- ▶ The Encoder-decoder model is used when we are taking an input sequence and translating it to an output sequence that is of a different length than the input, and doesn't align with it in a word-to-word way.
- ▶ example in part-of-speech tagging each token gets an associated tag, each input is associated with a specific output, and the labeling for that output takes mostly local information. Thus deciding whether a word is a verb or a noun, we look mostly at the word and the neighboring words.

The Encoder-Decoder Model with RNNs

- ▶ Encoder-decoder models are used especially for tasks like **machine translation**, where the input sequence and output sequence can have different lengths and the mapping between a token in the input and a token in the output can be very indirect (**in some languages the verb appears at the beginning of the sentence; in other languages at the end**).

Applications of encoder decoder network

- ▶ Encoder-decoder networks, sometimes called **sequence-to-sequence networks**, encoder-decoder are models capable of generating contextually appropriate, arbitrary length, output sequences given an input sequence. Encoder-decoder networks have been applied to a very wide range of applications including **summarization, question answering, and dialogue**, but they are particularly popular for machine translation.
- ▶ The key idea underlying these networks is the use of an encoder network that takes an input sequence and **creates a contextualized representation of it, often called the context**. This representation is then passed to a decoder which generates a task specific output sequence. Fig. 9.16 illustrates the architecture.

Encoder Decoder architecture

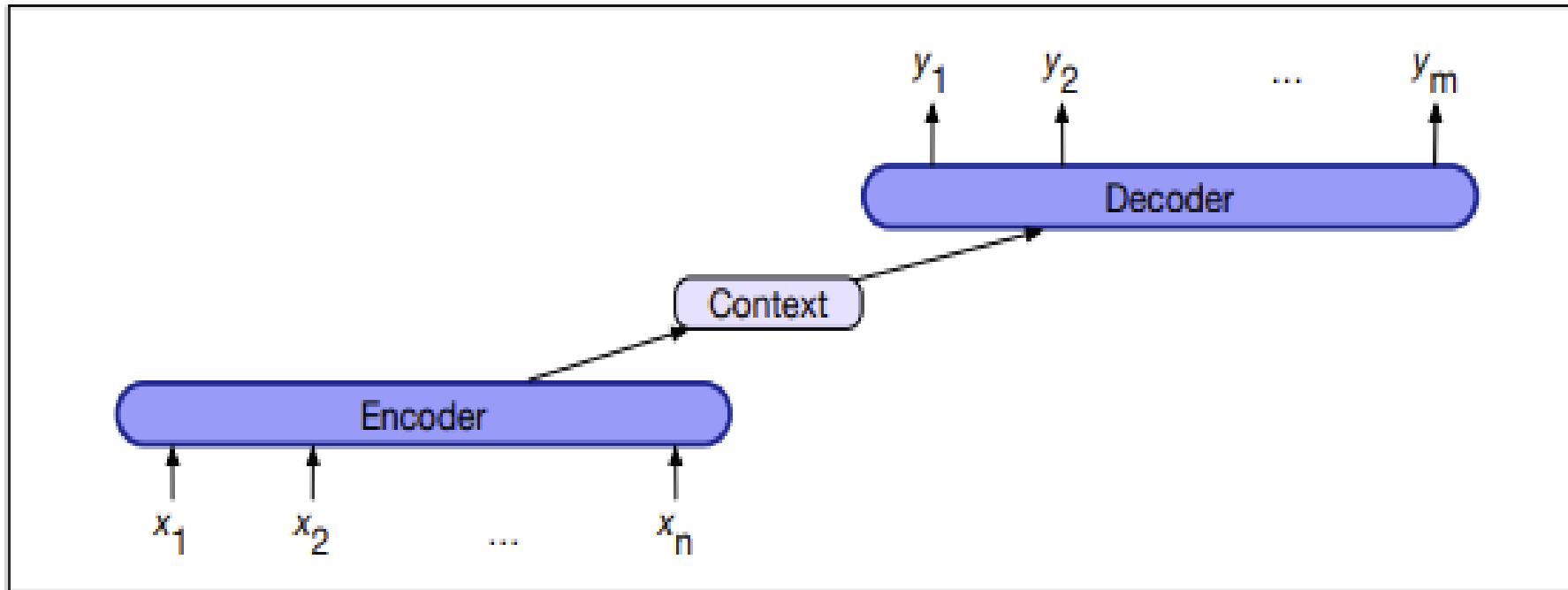


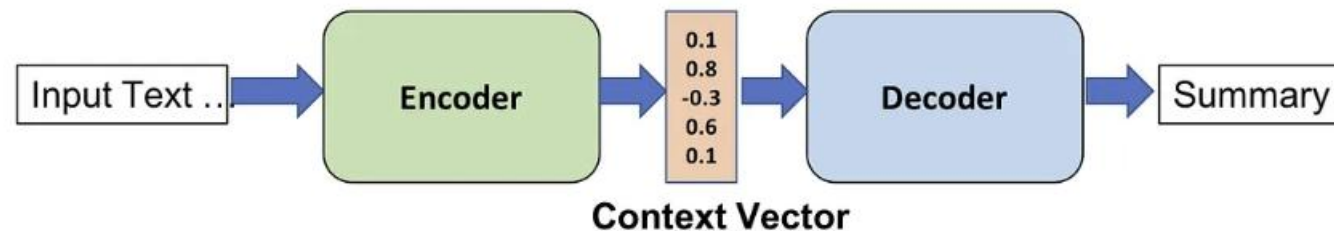
Figure 9.16 The encoder-decoder architecture. The context is a function of the hidden representations of the input, and may be used by the decoder in a variety of ways.

Encoder Decoder architecture

- ▶ Encoder-decoder networks consist of three conceptual components:
- ▶ 1. **An encoder** that accepts **an input sequence**, $x_{1:n}$, and generates a corresponding sequence of contextualized representations, $h_{1:n}$. LSTMs, convolutional networks, and transformers can all be employed as encoders.
- ▶ 2. **A context vector**, c , which is a function of $h_{1:n}$, and conveys the essence of the input to the decoder.
- ▶ 3. **A decoder**, which accepts c as input and generates an arbitrary length sequence of hidden states $h_{1:m}$, from which a corresponding sequence of output states $y_{1:m}$, can be obtained. Just as with encoders, decoders can be realized by any kind of sequence architecture.

Encoder Decoder architecture

- ▶ Encoding: The encoder processes the input sequence and compresses it into a fixed-length context vector.
- ▶ Decoding: The decoder uses the context vector to generate the output sequence, producing one token at a time until the end of the sequence is reached.



Encoder Decoder architecture

- Recall that in any language model, we can break down the probability as follows:

$$p(y) = p(y_1)p(y_2|y_1)p(y_3|y_1,y_2)\cdots p(y_m|y_1,\dots,y_{m-1}) \quad (9.28)$$

$$\mathbf{h}_t = g(\mathbf{h}_{t-1}, \mathbf{x}_t) \quad (9.29)$$

$$\mathbf{y}_t = f(\mathbf{h}_t) \quad (9.30)$$

We only have to make one slight change to turn this language model with autoregressive generation into an encoder-decoder model that is a **translation model that can translate from a source text in one language to a target text in a second**. add a sentence separation marker at the end of the source text, and then simply concatenate the target text.

Encoder Decoder architecture

- ▶ We could also illustrate encoder-decoder models with a question-answer pair, or a text-summarization pair.
- ▶ Let's use X to refer to the source text(English) plus the separator token $\langle s \rangle$, and Y to refer to the target text Y (in this case in Spanish). Then an encoder-decoder model computes the probability $p(y|x)$ as follows:

$$p(y|x) = p(y_1|x)p(y_2|y_1,x)p(y_3|y_1,y_2,x) \dots p(y_m|y_1, \dots, y_{m-1},x) \quad (9.31)$$

Encoder Decoder architecture

- ▶ Fig. 9.17 shows the setup for a simplified version of the encoder-decoder mode

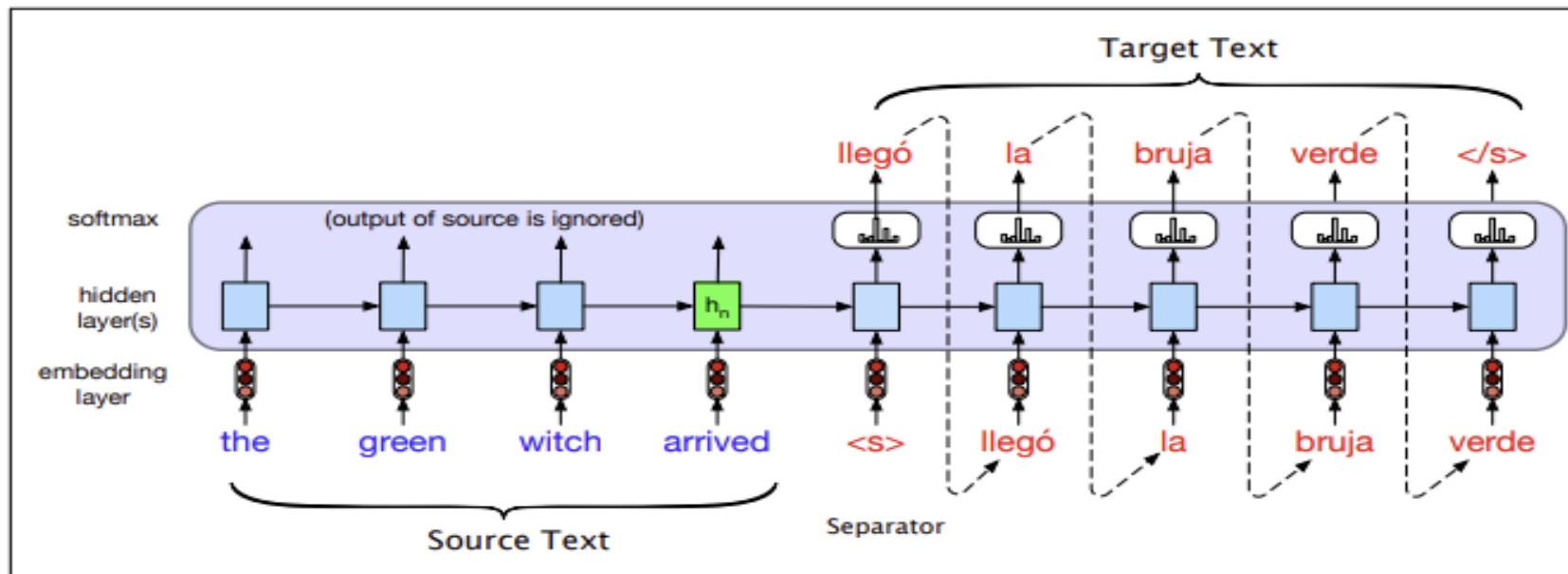


Figure 9.17 Translating a single sentence (inference time) in the basic RNN version of encoder-decoder approach to machine translation. Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

Encoder Decoder architecture

- ▶ Fig. 9.17 shows an English source text (“the green witch arrived”), a sentence separator token (<s>, and a Spanish target text (“llego la bruja verde”).
- ▶ To translate a source text, we run it through the network performing forward inference to generate hidden states until we get to the end of the source. Then we begin autoregressive generation, asking for a word in the context of the hidden layer from the end of the source input as well as the end-of-sentence marker. Subsequent words are conditioned on the previous hidden state and the embedding for the last word generated.

Encoder Decoder architecture

- ▶ In Fig. 9.18. The elements of the network on the left process the input sequence X and comprise the encoder. While our simplified figure shows only a single network layer for the encoder, stacked architectures are the norm, where the output states from the top layer of the stack are taken as the final representation, and the encoder consists of stacked biLSTMs where the hidden states from top layers from the forward and backward passes are concatenated to provide the contextualized representations for each time step.

Encoder Decoder architecture

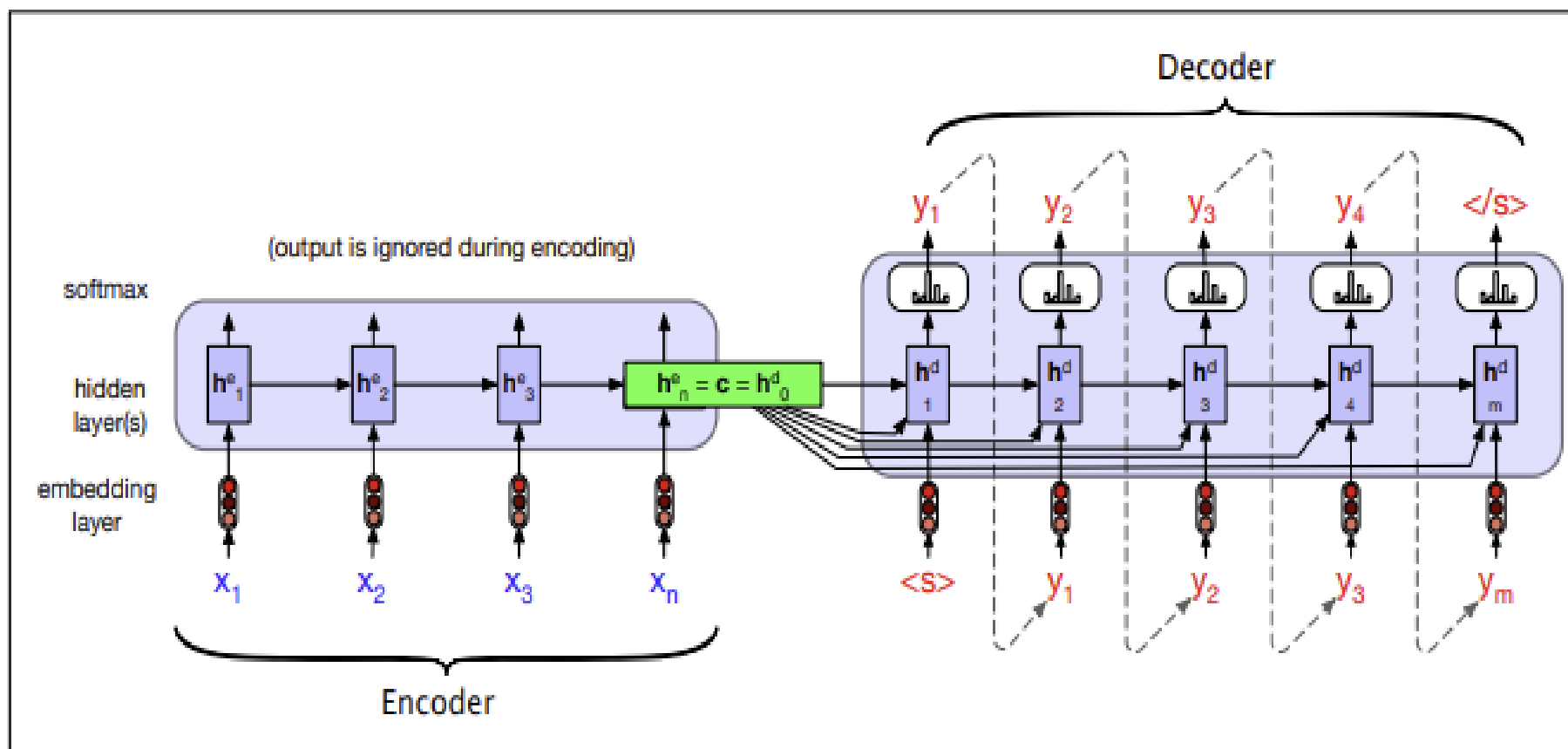


Figure 9.18 A more formal version of translating a sentence at inference time in the basic RNN-based encoder-decoder architecture. The final hidden state of the encoder RNN, h^e_n , serves as the context for the decoder in its role as h^d_0 in the decoder RNN, and is also made available to each decoder hidden state.

Encoder Decoder architecture

- ▶ The entire purpose of the encoder is to generate a contextualized representation of the input. This representation is embodied in the final hidden state of the encoder, h_n^e . This representation, also called C for context, is then passed to the decoder. The simplest version of the decoder network would take this state and use it just to initialize the first hidden state of the decoder; the first decoder RNN cell would use C as its prior hidden state h_0^d . The decoder would then autoregressively generate a sequence of outputs, one element at a time, until an end-of-sequence marker is generated. Each hidden state is conditioned on the previous hidden state and the output generated in the previous state.

Encoder Decoder architecture

- ▶ As Fig. 9.18 shows, we make the context vector $\mathbf{c}$ available to more than just the first decoder hidden state, to ensure that the influence of the context vector, $\mathbf{c}$, doesn't wane as the output sequence is generated. We do this by adding $\mathbf{c}$ as a parameter to the computation of the current hidden state. using the following equation:

$$\mathbf{h}_t^d = g(\hat{\mathbf{y}}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \quad (9.32)$$

Recall that g is a stand-in for some flavor of RNN and $\hat{\mathbf{y}}_{t-1}$ is the embedding for the output sampled from the softmax at the previous step:

$$\begin{aligned} \mathbf{c} &= \mathbf{h}_n^e \\ \mathbf{h}_0^d &= \mathbf{c} \\ \mathbf{h}_t^d &= g(\hat{\mathbf{y}}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \\ \mathbf{z}_t &= f(\mathbf{h}_t^d) \\ y_t &= \text{softmax}(\mathbf{z}_t) \end{aligned} \quad (9.33)$$

Training the Encoder-Decoder Model

- ▶ Encoder-decoder architectures are trained end-to-end. Each training example is a tuple of paired strings, a source and a target. Concatenated with a separator token, these source-target pairs can now serve as training data. For Machine Translation, the training data typically consists of sets of sentences and their translations. These can be drawn from standard datasets of aligned sentence pairs. Once we have a training set, the training itself proceeds as with any RNN-based language model. The network is given the source text and then starting with the separator token is trained autoregressively to predict the next word, as shown in Fig. 9.19

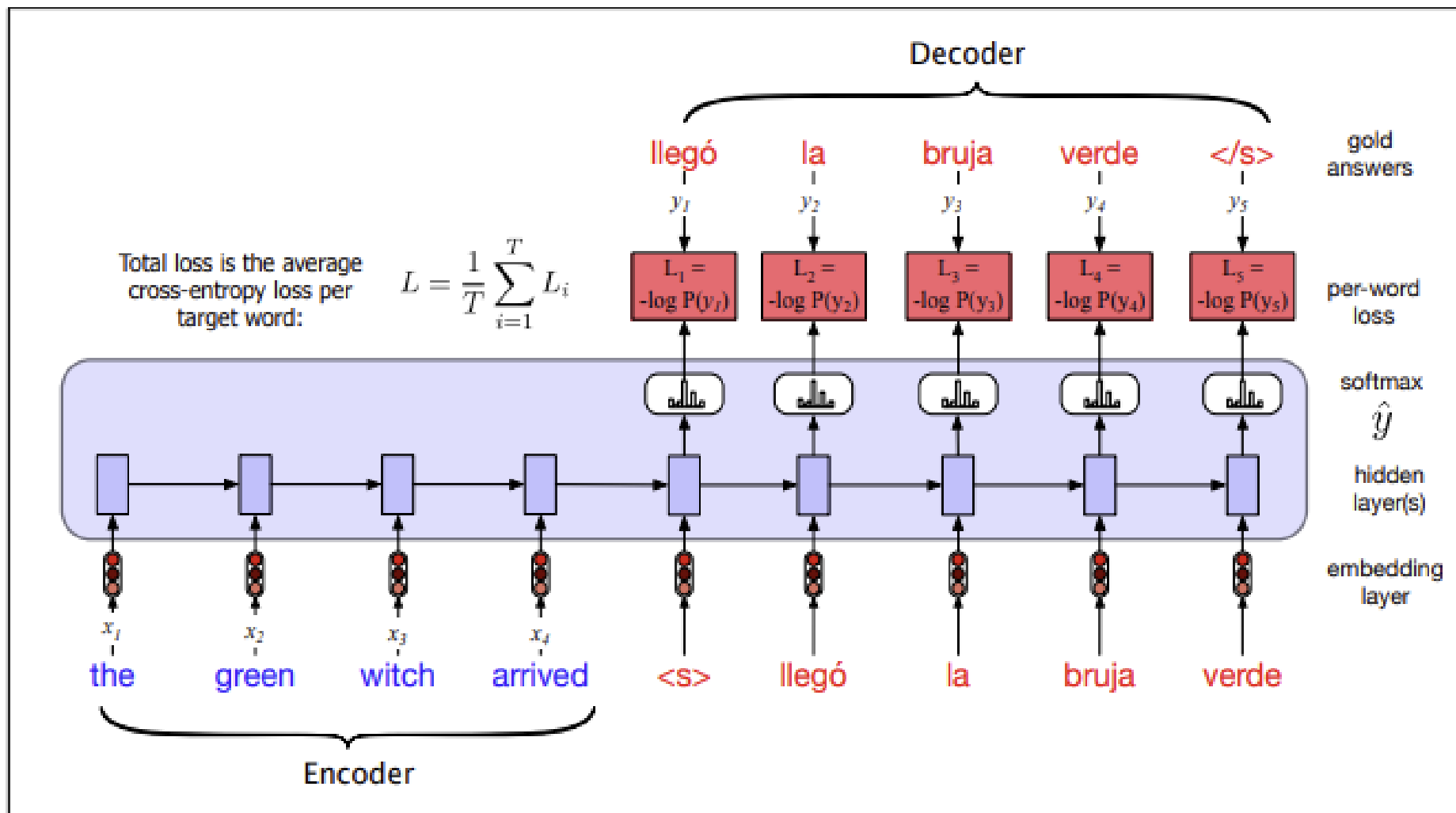


Figure 9.19 Training the basic RNN encoder-decoder approach to machine translation. Note that in the decoder we usually don't propagate the model's softmax outputs $\hat{y}_t$, but use **teacher forcing** to force each input to the correct gold value for training. We compute the softmax output distribution over $\hat{y}$ in the decoder in order to compute the loss at each token, which can then be averaged to compute a loss for the sentence.

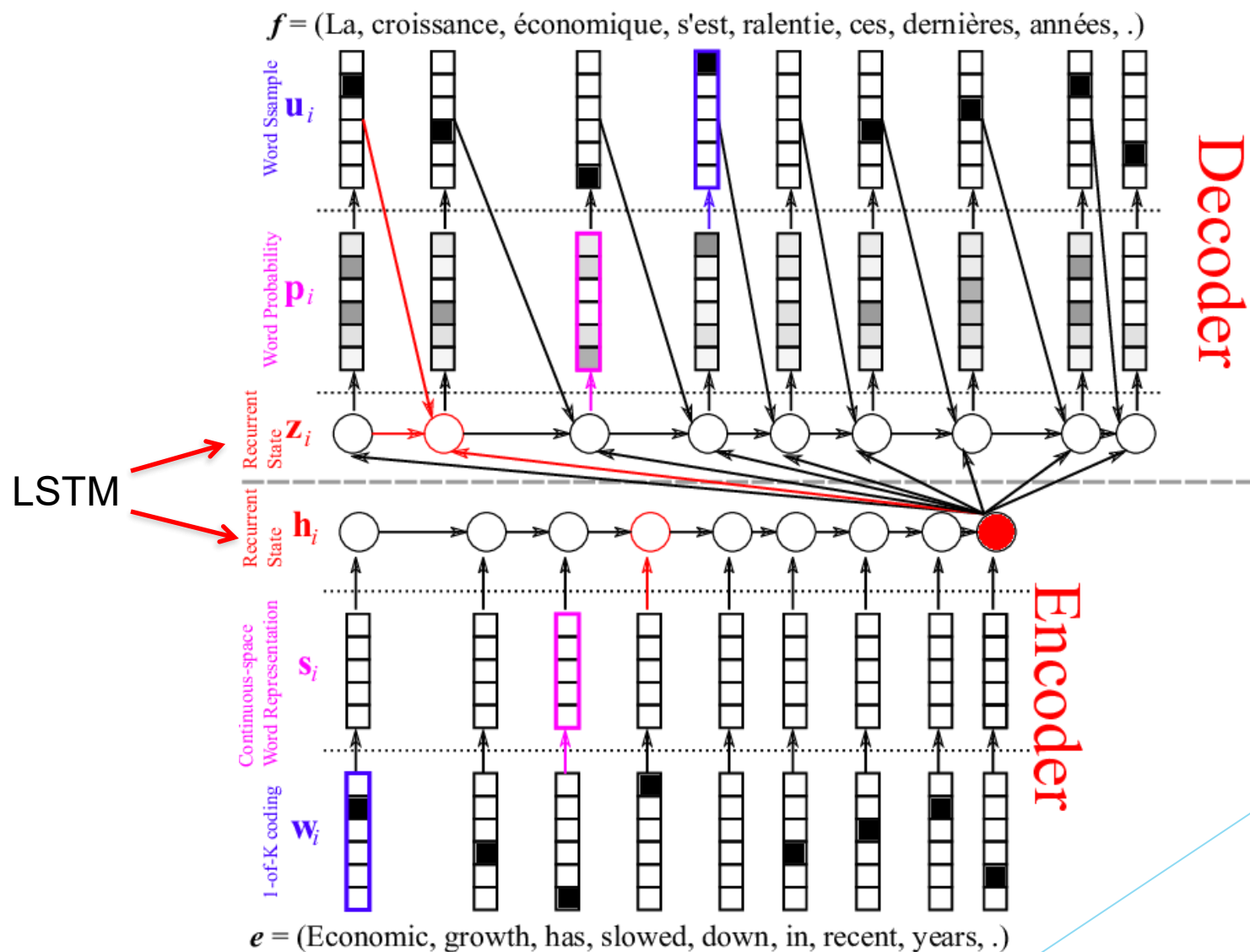
Training the Encoder-Decoder Model

- Note the differences between training (Fig. 9.19) and inference (Fig. 9.17) with respect to the outputs at each time step. The decoder during inference uses its own estimated output $\hat{Y}_t$ as the input for the next time step X_{t+1} . Thus the decoder will tend to deviate more and more from the gold target sentence as it keeps generating more tokens. In training, therefore, it is more common to use **teacher forcing** in the decoder. Teacher forcing means that we force the system to use the gold target token from training as the next input X_{t+1} , rather than allowing it to rely on the (possibly erroneous) decoder output $\hat{Y}_t$. This speeds up training.

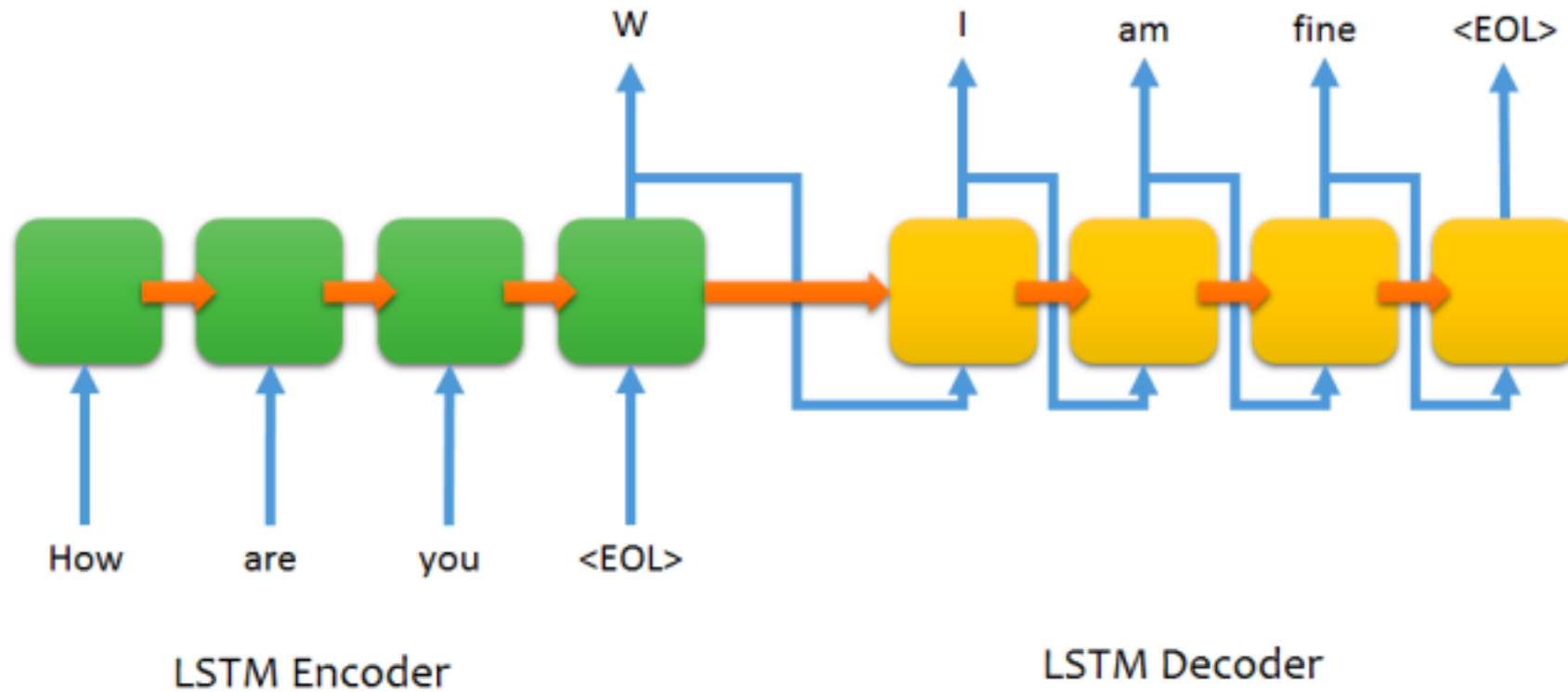
Applications of LSTM / RNN

- ▶ Neural machine translation
- ▶ Sequence to sequence chat model
- ▶ Chat with context

Neural machine translation



Sequence to sequence chat model

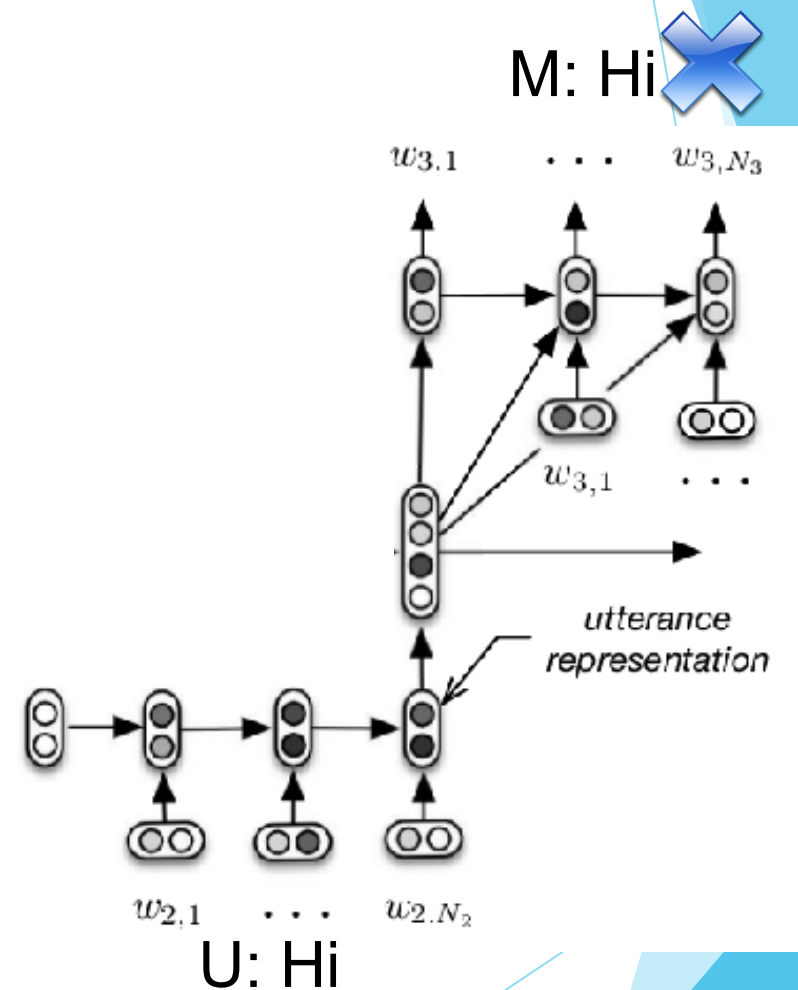


Chat with context

M: Hello

U: Hi

M: Hi



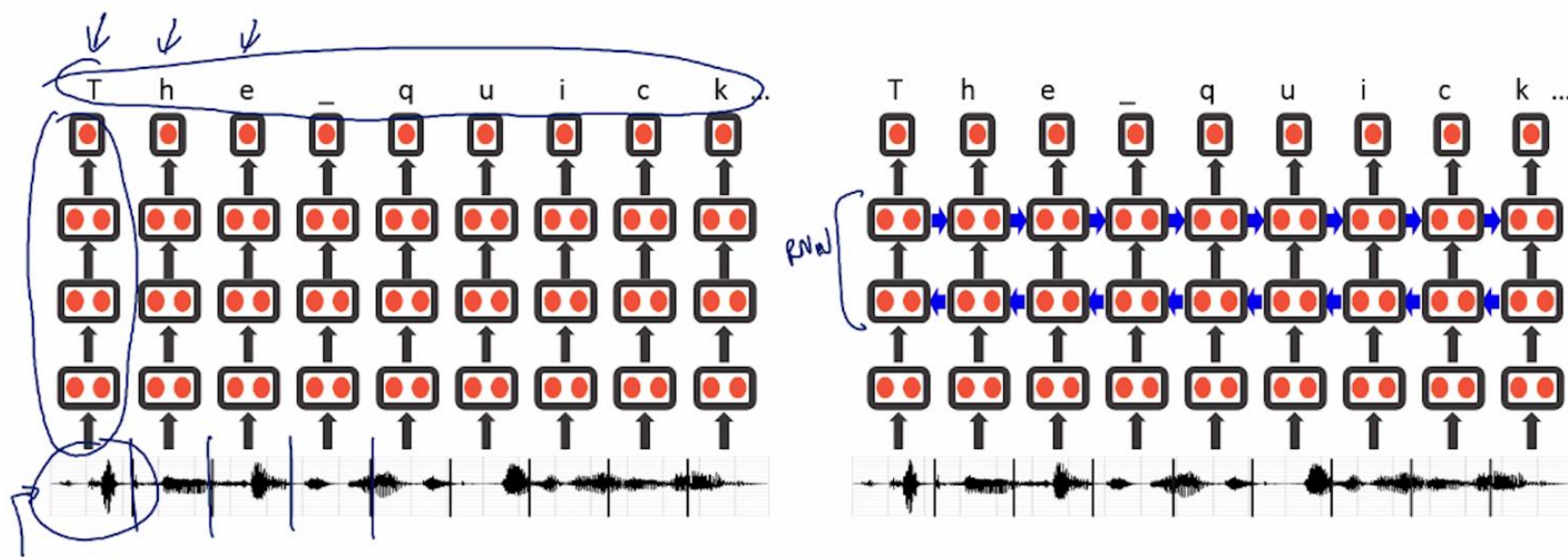
M: Hello

U: Hi

Serban, Iulian V., Alessandro Sordoni, Yoshua Bengio, Aaron Courville, and Joelle Pineau, 2015
"Building End-To-End Dialogue Systems Using Generative Hierarchical Neural Network Models."

Baidu's speech recognition using RNN

Speech recognition example (Deep Speech)



Limitations

- ▶ The fixed-length context vector can become a bottleneck for long input sequences, as it may struggle to capture all the necessary information.
- ▶ Basic encoder-decoder models do not have a mechanism to focus on specific parts of the input sequence during decoding.

Improvement

- ▶ **Attention Mechanism:** To address the limitations, the attention mechanism was introduced. It allows the decoder to selectively focus on relevant parts of the input sequence when generating each output token, significantly improving performance, especially for longer sequences.
- ▶ **Instead of relying solely on a single context vector, the decoder computes a weighted sum of the encoder's hidden states**, where the weights are determined by the relevance of each input state to the current decoding step.

Attention

- ▶ The simplicity of the encoder-decoder model is its clean separation of the encoder— which builds a representation of the source text—from the decoder, which uses this context to generate a target text. this context vector is h_n , the hidden state of the last (n th) time step of the source text. This final hidden state is thus acting as a bottleneck: it must represent absolutely everything about the meaning of the source text, since the only thing the decoder knows about the source text is what's in this context vector (Fig. 9.20). Information at the beginning of the sentence, especially for long sentences, may not be equally well represented in the context vector.

Attention

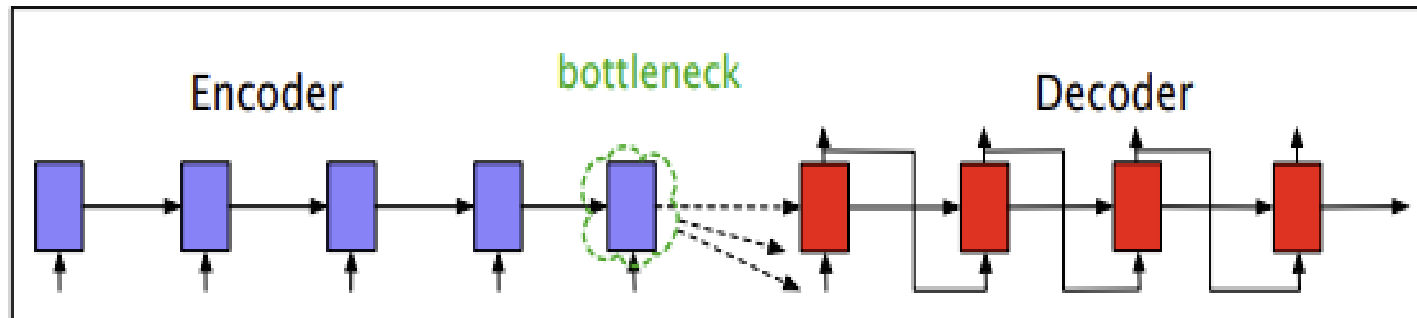
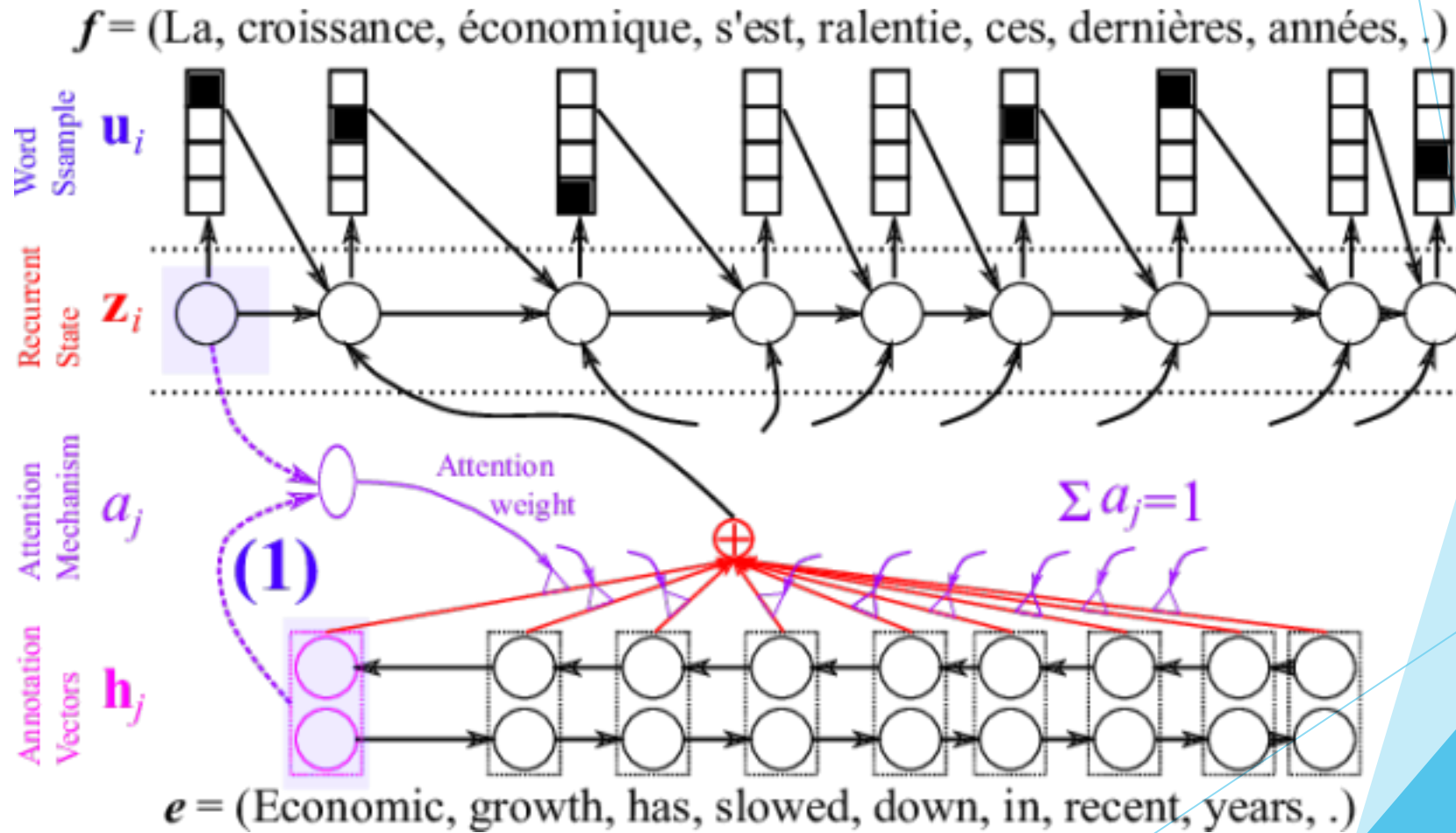


Figure 9.20 Requiring the context c to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.

The attention mechanism is a solution to the bottleneck problem, a way of allowing the decoder to get information from all the hidden states of the encoder, not just the last hidden state.

Attention



Attention

- ▶ In the attention mechanism, as in the vanilla encoder-decoder model, the context vector C is a single vector that is a function of the hidden states of the encoder, that is, $\mathbf{c} = f(\mathbf{h}_1^e \dots \mathbf{h}_n^e)$. Because the number of hidden states varies with the size of the input, we can't use the entire set of encoder hidden state vectors directly as the context for the decoder.
- ▶ The idea of attention is instead to create the single fixed-length vector C by taking a weighted sum of all the encoder hidden states. The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing. Attention thus replaces the static context vector with one that is dynamically derived from the encoder hidden states, different for each token in decoding.

Attention

- ▶ This context vector, $\mathbf{c}_i$, is generated a new with each decoding step i and takes all of the encoder hidden states into account in its derivation. We then make this context available during decoding by conditioning the computation of the current decoder hidden state on it, as we see in this equation (and Fig. 9.21):

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i) \quad (9.35)$$

Attention

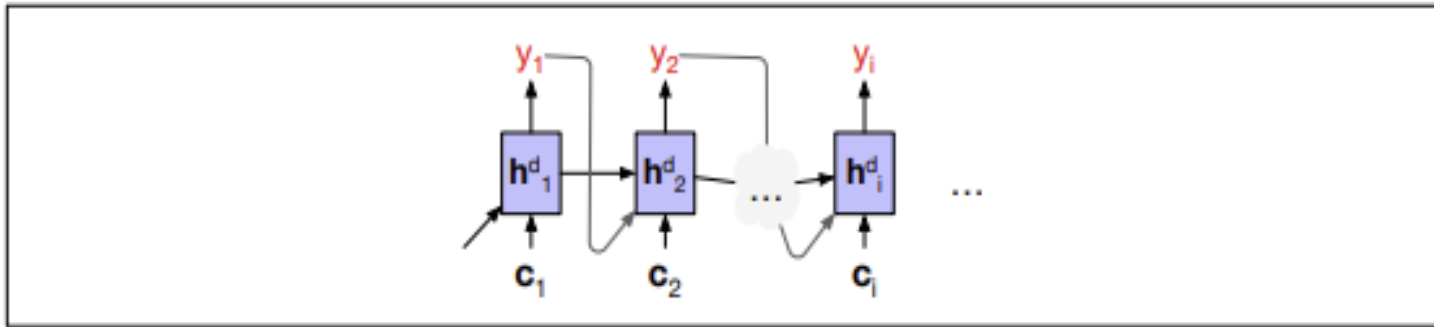


Figure 9.21 The attention mechanism allows each hidden state of the decoder to see a different, dynamic, context, which is a function of all the encoder hidden states.

The first step in computing c_i is to compute how much to focus on each encoder state, how relevant each encoder state is to the decoder state captured in h_{i-1}^d . We capture relevance by computing— at each state i during decoding—a $\text{score}(h_{i-1}^d, h_j^e)$ for each encoder state j .

Attention

- ▶ The simplest such score, called dot-product attention, implements relevance as **dot-product attention similarity**: measuring how similar the decoder hidden state is to an encoder hidden state, by computing the dot product between them:

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e \quad (9.36)$$

Attention

- ▶ The score that results from this dot product is a scalar that reflects the degree of similarity between the two vectors. The vector of these scores across all the encoder hidden states gives us the relevance of each encoder state to the current step of the decoder. To make use of these scores, we'll normalize them with a softmax to create a vector of weights, α_j , that tells us the proportional relevance of each encoder hidden state j to the prior hidden decoder state $\mathbf{h}_{i-1}^d$.

$$\begin{aligned}\alpha_j &= \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)) \\ &= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}\end{aligned}\tag{9.37}$$

Attention

- ▶ Finally, given the distribution in α , we can compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states.

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e \quad (9.38)$$

With this, we finally have a fixed-length context vector that takes into account information from the entire encoder state that is dynamically updated to reflect the needs of the decoder at each step of decoding. Fig. 9.22 illustrates an encoder-decoder network with attention, focusing on the computation of one context vector $\mathbf{c}_i$.

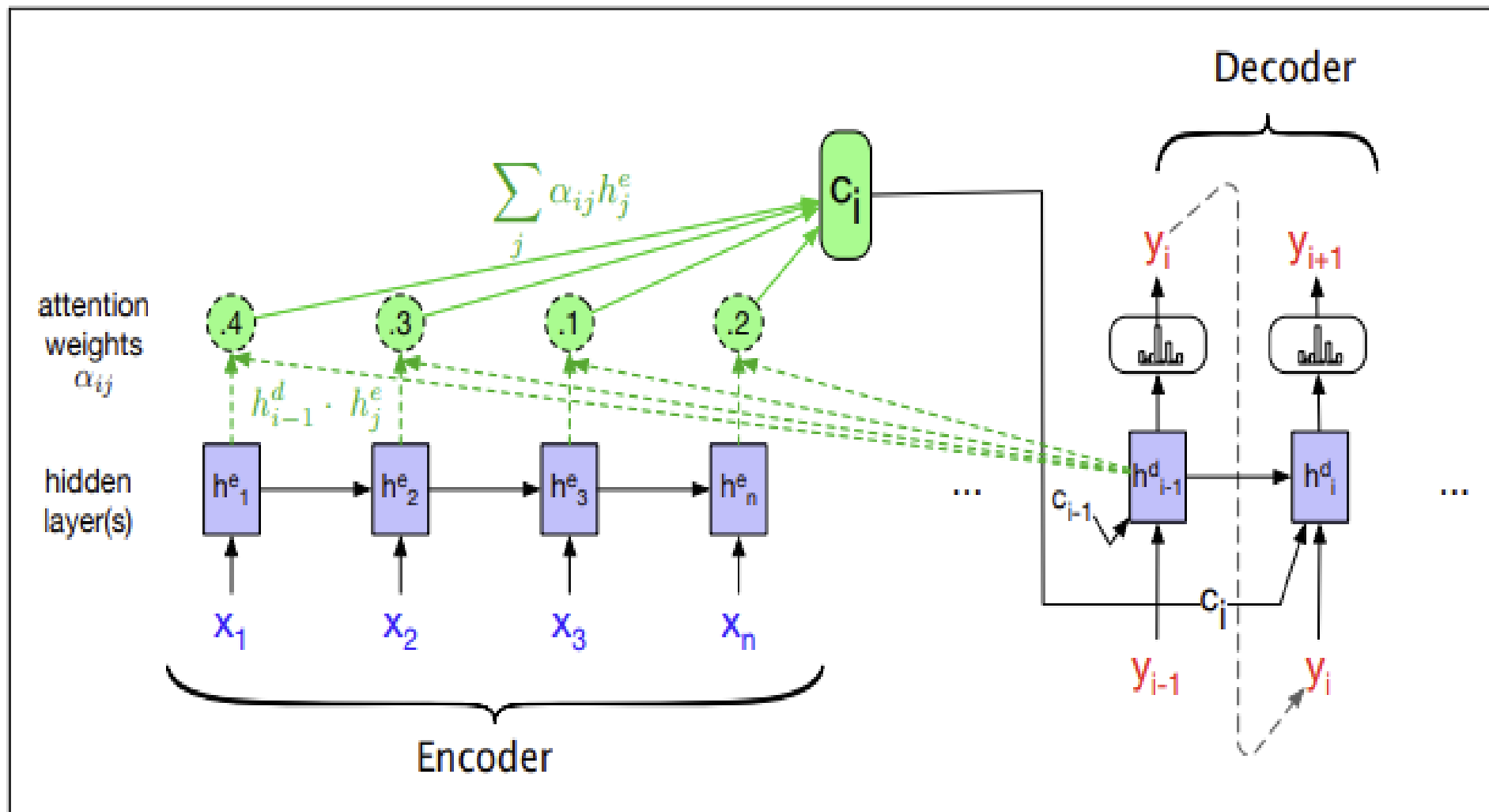


Figure 9.22 A sketch of the encoder-decoder network with attention, focusing on the computation of c_i . The context value c_i is one of the inputs to the computation of h^d_i . It is computed by taking the weighted sum of all the encoder hidden states, each weighted by their dot product with the prior decoder hidden state h^d_{i-1} .

Mathematical representation

Instead of one static context vector C , we calculate a unique context vector c_i for **every** output step i .

1. **Alignment Scores:** For a specific decoder step, we compare the current decoder hidden state s_{i-1} with *all* encoder hidden states $h_1, \dots, h_n$.
2. **Softmax Weights:** We turn these scores into probabilities (weights α) that sum to 1.
3. **Weighted Sum:** We multiply the encoder hidden states by these weights to get the context vector.

- Equation: $c_i = \sum_{j=1}^n \alpha_{ij} h_j$

Types of Attention

- ▶ **Bahdanau (Additive) vs. Luong (Multiplicative)**
- ▶ **Bahdanau Attention:** The "original" attention. The alignment score is calculated using a small neural network layer. It is often more computationally expensive but very flexible.
- ▶ **Luong Attention:** Uses a simple dot-product between hidden states. It is faster and more memory-efficient.

Types of Attention

- ▶ **Global vs. Local Attention**
- ▶ **Global:** Considers every single hidden state in the encoder.
- ▶ **Local:** Focuses only on a small "window" of hidden states around the current position, which is useful for extremely long documents.

The Limitations of RNN-based Attention

- ▶ Even with attention, RNNs are slow because they are **sequential**.
- ▶ We cannot compute step 100 until you have computed steps 1 through 99. This prevents massive parallelization on GPUs.

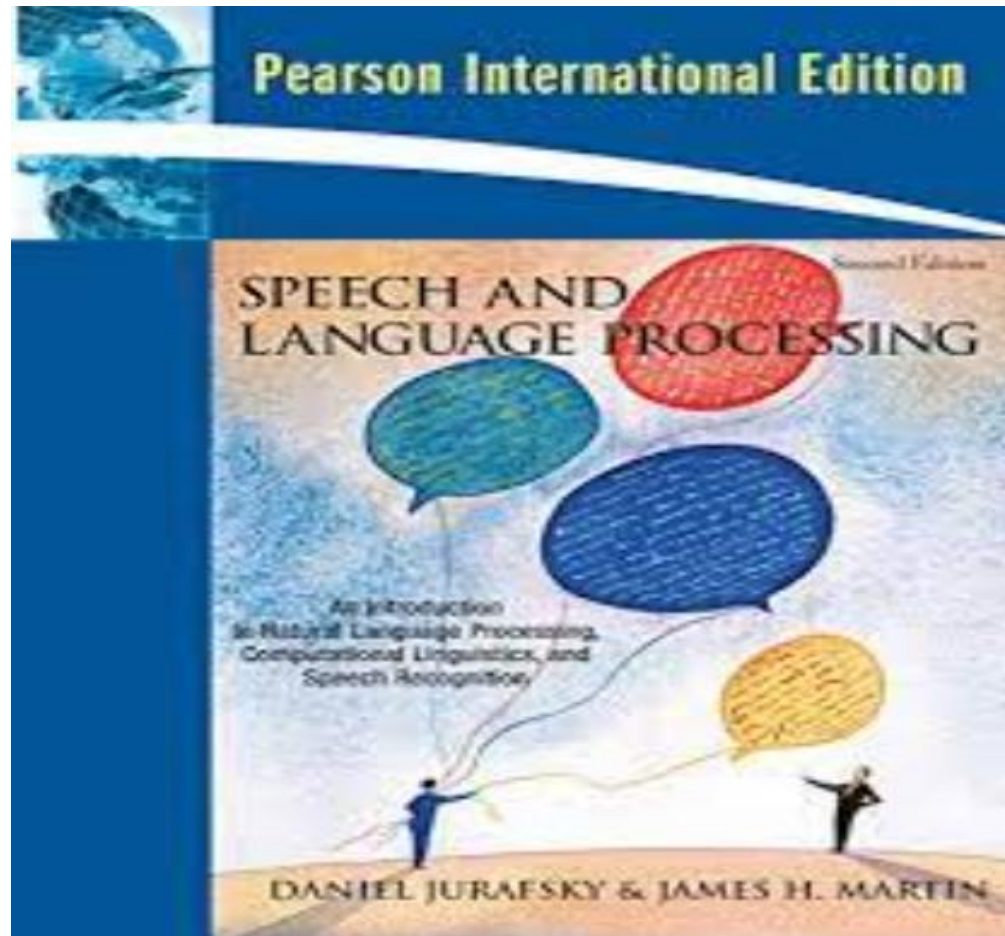
Summary

- ▶ In simple Recurrent Neural Networks sequences are processed one element at a time, with the output of each neural unit at time t based both on the current input at t and the hidden layer from time $t-1$.
- ▶ RNNs can be trained with a straightforward extension of the backpropagation algorithm, known as backpropagation through time (BPTT).
- ▶ Simple recurrent networks fail on long inputs because of problems like vanishing gradients; instead modern systems use more complex gated architectures such as LSTMs that explicitly decide what to remember and forget in their hidden and context layers.

Summary

- ▶ Common language-based applications for RNNs include:
 - Probabilistic language modeling: assigning a probability to a sequence, or to the next element of a sequence given the preceding words.
 - Auto-regressive generation using a trained language model.
 - Sequence labeling like part-of-speech tagging, where each element of a sequence is assigned a label
 - Sequence classification, where an entire text is assigned to a category, as in spam detection, sentiment analysis or topic classification.
 - Encoder-decoder architectures, where an input is mapped to an output of different length and alignment.

Reference



Chapter 9

Question



Thank you

